

Slack風アプリ 検索機能 詳細設計

JSON対応・DB定義準拠 / GitHubで文字化けしにくいPDF

本資料は、送付されたDB定義画像を前提に、検索機能のIF定義、JSON設計、処理フロー、SQL例、DTO、JavaScriptの役割を整理したものです。

PDF内の日本語は Noto Sans CJK JP 系フォントを使って作成しています。ファイル名も英数字にしているため、GitHubに置いたときの文字化けリスクを下げています。

1. 前提・設計方針

- 検索処理は JavaScript で検索条件をJSON化し、ServletへPOST送信する。
- ServletはJSONをDTOへ変換し、RepositoryでDB検索を行う。
- 検索結果はJSONでJavaScriptへ返し、画面の検索結果エリアだけを動的に更新する。
- DB画像に合わせ、テーブル名は message / channel / user_channel / channel_message / user_message / search を使用する。
- IDは CHAR(6 CHAR) のため、JavaとJSONでは long ではなく String として扱う。
- user_workspace は削除する流れのため、本設計では検索処理に使わない。ワークスペース所属は user.workspace_id と channel.workspace_id で確認する。

2. 使用テーブル

テーブル名	検索機能での役割
message	メッセージ本文 message_text、作成日時 created_at を検索対象にする。
channel_message	message と channel を紐づける中間テーブル。
channel	チャンネル名、workspace_id、公開状態 channel_status を取得する。
user_message	message と投稿者 user を紐づける中間テーブル。
user	投稿者名 display_name、ログインユーザーの workspace_id を確認する。
user_channel	ログインユーザーがそのチャンネルに参加しているか確認する。
search	検索キーワードを履歴として保存する場合に使用する。検索結果取得自体には必須ではない。

3. IF定義

3.1 検索画面表示IF

項目	内容
処理名	検索画面表示
URL	/search
HTTPメソッド	GET
Servlet	SearchServlet#doGet
JSP	search.jsp
主な処理	ログイン確認、チャンネル一覧取得、検索画面表示

3.2 検索実行IF

項目	内容
処理名	メッセージ検索
URL	/search
HTTPメソッド	POST
Content-Type	application/json; charset=UTF-8
Servlet	SearchServlet#doPost
送信形式	JSON
返却形式	JSON

DB検索対象	message.message_text
--------	----------------------

4. リクエストJSON定義

IDはDB定義上 CHAR(6 CHAR) のため、JSONでは数値ではなく文字列で送る。

```
{
  "workspaceId": "000001",
  "keyword": "会議",
  "channelId": "000003",
  "senderId": "000005",
  "startDate": "2026-06-01",
  "endDate": "2026-06-30",
  "sort": "new",
  "page": 1
}
```

項目名	型	必須	内容
workspaceId	String	必須	検索対象ワークスペースID。CHAR(6)で扱う。
keyword	String	任意	検索キーワード。message.message_text に対してLIKE検索する。
channelId	String	任意	特定チャンネルに絞り込む場合に指定する。
senderId	String	任意	投稿者で絞り込む場合に指定する。
startDate	String	任意	検索開始日。形式は yyyy-MM-dd。
endDate	String	任意	検索終了日。形式は yyyy-MM-dd。
sort	String	任意	new または old。未指定時は new。
page	int	任意	ページ番号。未指定時は 1。

5. レスポンスJSON定義

```
{
  "success": true,
  "resultCount": 2,
  "messageList": [
    {
      "messageId": "000101",
      "channelId": "000003",
      "channelName": "general",
      "senderId": "000005",
      "senderName": "田中",
      "messageText": "明日の会議は10時からです",
      "createdAt": "2026-06-10 09:30:00"
    }
  ],
  "errorMessage": null
}
```

項目名	型	内容
success	boolean	検索成功なら true。エラー時は false。
resultCount	int	検索結果件数。
messageList	array	検索結果の一覧。
errorMessage	String	エラー時の表示メッセージ。成功時は null。

messageListの中身

項目名	型	内容
messageId	String	メッセージID。

channelId	String	チャンネルID。
channelName	String	チャンネル名。
senderId	String	投稿者ユーザーID。
senderName	String	投稿者表示名。
messageText	String	メッセージ本文。
createdAt	String	投稿日時。画面表示用に文字列化する。

6. バリデーション定義

項目	チェック内容	エラー時の扱い
workspaceId	必須、6桁、ログインユーザーの所属workspaceと一致すること。	不正なアクセスです。
keyword	255文字以内。空文字は null として扱う。	キーワードは255文字以内で入力してください。
channelId	指定時は6桁。ログインユーザーが user_channel に存在すること。	指定されたチャンネルを検索できません。
senderId	指定時は6桁。対象workspace内のユーザーであること。	投稿者指定が不正です。
startDate	指定時は yyyy-MM-dd 形式。	開始日の形式が不正です。
endDate	指定時は yyyy-MM-dd 形式。	終了日の形式が不正です。
startDate / endDate	開始日が終了日以前であること。	開始日は終了日以前の日付を指定してください。
sort	new または old。未指定や不正値は new にする。	画面にはエラーを出さず新しい順で検索する。
page	1以上の整数。未指定時は1。	1ページ目として扱う。

7. 処理フロー

7.1 検索画面表示

- SearchServlet#doGet が呼ばれる。
- セッションから loginUser を取得する。未ログインならログイン画面へ戻す。
- ログインユーザーの workspace_id を取得する。
- user_channel と channel を使い、ログインユーザーが参加しているチャンネル一覧を取得する。
- search.jsp にフォワードし、検索画面を表示する。

7.2 検索実行

- search.js が画面入力値を取得し、検索条件をJSONにする。
- fetch で SearchServlet#doPost へPOST送信する。
- ServletでJSONを SearchConditionDto に変換する。
- Serviceでバリデーションと権限チェックを行う。
- Repositoryで message / channel_message / channel / user_message / user / user_channel を結合して検索する。
- 検索キーワードがある場合、必要に応じて search テーブルへ検索履歴を登録する。
- SearchResponseDto をJSONへ変換して返却する。
- search.js が検索結果エリアを更新する。

8. DB検索の考え方

このDBでは message テーブルに channel_id や user_id が直接存在しないため、中間テーブルを経由する。

```

チャンネル情報の取得:
message -> channel_message -> channel

投稿者情報の取得:
message -> user_message -> user

閲覧権限の確認:
channel -> user_channel -> loginUser.id

```

user_workspace は削除する前提なので、検索では使用しない。ログインユーザーが所属する workspace は user.workspace_id を使って確認する。

9. SQL例

Oracle想定SQL例。keyword、channelId、senderId、startDate、endDate は、指定がある場合だけWHERE句へ追加する動的SQLにすると書きやすい。

```

SELECT
  m.message_id,
  m.message_text,
  m.created_at,
  c.channel_id,
  c.channel_name,
  u.id AS sender_id,
  u.display_name AS sender_name
FROM
  message m
INNER JOIN channel_message cm
  ON m.message_id = cm.message_id
INNER JOIN channel c
  ON cm.channel_id = c.channel_id
INNER JOIN user_message um
  ON m.message_id = um.message_id
INNER JOIN "user" u
  ON um.id = u.id
INNER JOIN user_channel uc
  ON c.channel_id = uc.channel_id
WHERE
  c.workspace_id = ?
AND uc.id = ?
AND c.workspace_id = (
  SELECT workspace_id
  FROM "user"
  WHERE id = ?
)
/* keywordがある場合 */
AND m.message_text LIKE '%' || ? || '%'
/* channelIdがある場合 */
AND c.channel_id = ?
/* senderIdがある場合 */
AND u.id = ?
/* startDateがある場合 */
AND m.created_at >= TO_DATE(?, 'YYYY-MM-DD')
/* endDateがある場合 */
AND m.created_at < TO_DATE(?, 'YYYY-MM-DD') + 1
ORDER BY
  m.created_at DESC
OFFSET ? ROWS FETCH NEXT ? ROWS ONLY

```

注意: user はDBによって予約語扱いになる可能性があるため、SQLでは "user" のように囲むか、app_user などにテーブル名変更するのが安全。

10. 検索履歴登録

search テーブルは検索結果を取得するための必須テーブルではなく、検索履歴を保存するために使う。keyword が空の場合は登録しない。

```
INSERT INTO search (
  search_id,
  id,
  search_text
) VALUES (
  LPAD(search_seq.NEXTVAL, 6, '0'),
  ?,
  ?
)
```

カラム名	設定値
search_id	search_seq から採番した6桁文字列。
id	検索したログインユーザーID。
search_text	検索キーワード。最大255文字。

11. DTO定義

11.1 SearchConditionDto

```
public class SearchConditionDto {
  private String workspaceId;
  private String keyword;
  private String channelId;
  private String senderId;
  private String startDate;
  private String endDate;
  private String sort;
  private int page;

  // getter / setter
}
```

11.2 SearchResultDto

```
public class SearchResultDto {
  private String messageId;
  private String channelId;
  private String channelName;
  private String senderId;
  private String senderName;
  private String messageText;
  private String createdAt;

  // getter / setter
}
```

11.3 SearchResponseDto

```
import java.util.List;

public class SearchResponseDto {
  private boolean success;
  private int resultCount;
  private List<SearchResultDto> messageList;
  private String errorMessage;

  // getter / setter
}
```

12. JavaScript設計

JSPは画面の土台、JavaScriptはJSON送信と結果表示、Servletは検索処理の入口として役割を分ける。

```

async function searchMessages() {
  const condition = {
    workspaceId: document.getElementById("workspaceId").value,
    keyword: document.getElementById("keyword").value || null,
    channelId: document.getElementById("channelId").value || null,
    senderId: document.getElementById("senderId").value || null,
    startDate: document.getElementById("startDate").value || null,
    endDate: document.getElementById("endDate").value || null,
    sort: document.getElementById("sort").value || "new",
    page: 1
  };

  const response = await fetch("search", {
    method: "POST",
    headers: { "Content-Type": "application/json; charset=UTF-8" },
    body: JSON.stringify(condition)
  });

  const data = await response.json();
  displaySearchResult(data);
}

```

検索結果表示

```

function displaySearchResult(data) {
  const area = document.getElementById("searchResultArea");
  area.innerHTML = "";

  if (!data.success) {
    area.textContent = data.errorMessage || "検索に失敗しました。";
    return;
  }

  if (data.resultCount === 0) {
    area.textContent = "条件に一致するメッセージはありません。";
    return;
  }

  data.messageList.forEach(message => {
    const div = document.createElement("div");
    div.className = "search-result";
    div.innerHTML = `
      <p><strong>#${message.channelName}</strong>
        ${message.senderName} ${message.createdAt}</p>
      <p>${message.messageText}</p>
      <a href="channel?channelId=${message.channelId}&messageId=${message.messageId}">
        メッセージへ移動
      </a>`;
    area.appendChild(div);
  });
}

```

13. Servlet設計

処理	内容
文字コード設定	<code>request.setCharacterEncoding("UTF-8")</code> を設定する。
レスポンス形式	<code>application/json; charset=UTF-8</code> を設定する。
ログイン確認	セッションの <code>loginUser</code> を確認する。
JSON読込	<code>request.getReader()</code> で本文を読み込む。
DTO変換	Gsonなどで <code>SearchConditionDto</code> に変換する。
検索呼出	<code>SearchService.searchMessages(loginUserId, condition)</code> を呼ぶ。
JSON返却	<code>SearchResponseDto</code> をJSON化して <code>response.getWriter().write()</code> で返す。

```
protected void doPost(HttpServletRequest request,
                      HttpServletResponse response)
    throws ServletException, IOException {

    request.setCharacterEncoding("UTF-8");
    response.setContentType("application/json; charset=UTF-8");

    HttpSession session = request.getSession(false);
    if (session == null || session.getAttribute("loginUser") == null) {
        response.getWriter().write(
            "{\"success\":false,\"errorMessage\":\"ログインしてください\"}");
    };
    return;
}

// JSON読み込 -> DTO変換 -> Service呼出 -> JSON返却
}
```

14. ページング設計

項目	内容
1ページ件数	20件
page未指定	1ページ目として扱う。
offset計算	(page - 1) * 20
SQL	OFFSET ? ROWS FETCH NEXT ? ROWS ONLY

```
int limit = 20;
int page = condition.getPage() <= 0 ? 1 : condition.getPage();
int offset = (page - 1) * limit;
```

15. エラーメッセージ定義

エラー内容	表示メッセージ
未ログイン	ログインしてください。
workspaceId不正	不正なアクセスです。
キーワード文字数超過	キーワードは255文字以内で入力してください。
日付形式不正	日付の形式が正しくありません。
開始日 > 終了日	開始日は終了日以前の日付を指定してください。
チャンネル権限なし	指定されたチャンネルを検索できません。
検索結果0件	条件に一致するメッセージはありません。
DBエラー	検索処理中にエラーが発生しました。

16. GitHubで文字化けさせないための注意

- PDFは日本語フォントを埋め込んで作成する。今回は Noto Sans CJK JP 系フォントを使用。
- PDFファイル名は日本語ではなく search_design_git_safe.pdf のように英数字にする。
- Gitに上げるときはPDFをバイナリファイルとして扱う。中身をテキスト変換しない。
- READMEにリンクを書く場合は、README.md をUTF-8で保存する。
- GitHubのプレビューで崩れる場合でも、PDF自体はダウンロードして開けば正しく表示されることがある。

README.md に書く例:

[検索機能 詳細設計PDF](./search_design_git_safe.pdf)

17. まとめ

今回の検索機能は、以下の構成で作るとDB定義と合いやすい。

```
search.jsp
-> search.js
-> SearchServlet
-> SearchService
-> SearchRepository
-> DB
-> JSON response
-> search.jsで画面更新
```

- 検索対象は message.message_text。
- チャンネル情報は channel_message と channel から取得する。
- 投稿者情報は user_message と user から取得する。
- 閲覧権限は user_channel で確認する。
- user_workspace は使わない。
- IDは CHAR(6) なので String で扱う。